

Infinite Connections

Comprehensive Study Guide

AI-Generated NYT Connections Puzzles

STA 561D -- Duke University

Spring 2026

Table of Contents

1. The Connections Puzzle
2. Foundation: Sentence Embeddings and Cosine Similarity
3. Puzzle Generation Pipeline
4. Solver Algorithms
5. Roundtable Validation
6. Evaluation Metrics
7. System Architecture
8. Key Formulas Reference Card
9. What Is Original vs. From the Literature
10. References

1. The Connections Puzzle

What Is It?

NYT Connections is a daily word puzzle published by The New York Times. The player is given 16 words and must sort them into 4 groups of 4, where each group shares a hidden connection. Groups are color-coded by difficulty:

Yellow	Easiest -- e.g., WAYS TO SAY HELLO: Hi, Hey, Howdy, Yo
Green	Medium -- e.g., PLANETS: Mars, Venus, Saturn, Jupiter
Blue	Hard -- e.g., FIRE ____: Work, Fly, Place, Side
Purple	Hardest -- e.g., TAYLOR SWIFT ALBUMS: Folklore, Reputation, ...

Players get 4 mistakes before losing. A guess of 3/4 correct triggers 'One away!' feedback.

Why Is Generation Hard?

- Unique solution: 2,627,625 possible groupings, exactly one must be correct.
- Difficulty gradient: groups must range from obvious to subtle.
- Deception: good puzzles contain plausible-but-wrong 'false groups.'
- Diversity: categories span synonyms, wordplay, fill-in-the-blank, pop culture, etc.

The NYT Dataset

We work with a ground truth dataset of 554 NYT Connections puzzles (June 2023 - December 2024).

Metric	Value
Total puzzles	554
Unique words	4,918
Unique categories	2,054
Multi-word entries	112
Puzzles with difficulty	227 / 554
Difficulty mean +/- std	2.94 +/- 0.64
Most frequent word	BALL (15 appearances)

Category type distribution across 2,216 slots: Common property 46%, Synonyms/Slang 38%, Fill-in-the-blank 9%, Pop culture 6%, Wordplay 1%.

2. Foundation: Sentence Embeddings and Cosine Similarity

Every component in this project relies on sentence embeddings -- dense vector representations of words that capture semantic meaning.

The MPNET Model

We use all-mpnet-base-v2 from sentence-transformers. It maps each word to a 768-dimensional vector. Words with similar meanings have vectors pointing in similar directions: 'PIANO' and 'GUITAR' are close; 'PIANO' and 'TRUCK' are far apart.

Cosine Similarity

Cosine Similarity

$$\cos(a, b) = (a \cdot b) / (||a|| * ||b||)$$

Range: $[-1, 1]$, where 1 = identical direction, 0 = orthogonal

Average Pairwise Similarity

For a group of 4 words, there are $C(4,2) = 6$ pairs:

Average Pairwise Similarity

$$s_{avg} = (1/6) * \text{SUM over all } i < j \text{ of } \cos(e_i, e_j)$$

Higher s_{avg} = words are more semantically related = easier group.

Key Idea

Average pairwise cosine similarity is the fundamental building block. It is used for: (1) selecting the best 4 words from 8 candidates, (2) scoring groups in the embedding solver, (3) assigning difficulty colors, and (4) evaluating puzzle quality.

3. Puzzle Generation Pipeline

The pipeline creates puzzles through six stages. The LLM provides creativity; MPNET provides consistency.

Stage 1: Group Creation (LLM + Embedding Selection)

Each puzzle is built one group at a time across 4 LLM calls:

1. Sample 4 random seed words from the NYT word bank
2. LLM generates a category name + 8 candidate words (story-injected)
3. MPNET evaluates all $C(8,4) = 70$ subsets and picks the best 4
4. Remove any words already used in previous groups
5. Repeat for all 4 groups

Key Idea

The LLM is NOT allowed to pick the final 4 words. LLMs are bad at estimating semantic similarity and make inconsistent selections. MPNET selection is deterministic and correlates with human difficulty perception.

Story Injection for Diversity

Without intervention, the LLM generates the same ~20 categories repeatedly. The story injection technique solves this:

Prompt: 'First, write a short story (2-3 sentences) using these words: [4 random words]. Then use that story as inspiration for your category.'

The random seed words force the LLM into different creative directions each time.

Stage 2: False-Group Method (Primary)

This produces the highest-quality puzzles. A user study found that false-group puzzles beat NYT originals in user preference 42.86% of the time.

Algorithm:

1. LLM generates a 'root group' of 4 words, each with an alternate meaning
2. For each root word, generate a NEW category inspired by the alternate meaning
3. MPNET selects best 4 from 8 candidates for each new category
4. The 4 new categories = the real puzzle; the root group = discarded decoy

Example: Root = 'THINGS THAT ARE ROUND' -> {BALL, GLOBE, RING, WHEEL}

BALL -> 'formal dance event' -> generates DANCE category

GLOBE -> 'theater or news org' -> generates THEATER category

RING -> 'sound a phone makes' -> generates SOUNDS category

WHEEL -> 'to turn or rotate' -> generates MOVEMENT category

The solver sees words that LOOK like 'ROUND THINGS', but that group doesn't exist.

Stage 3: Editor Pass

A second LLM call reviews the puzzle and fixes inaccurate category names.

Stage 4: Difficulty Color Assignment

Color Assignment Rule

1. Compute avg pairwise cosine similarity for each group
2. Sort groups by similarity DESCENDING
3. Assign: highest -> Yellow, next -> Green, next -> Blue, lowest -> Purple

Intuition: high similarity = words obviously relate = easy. Low similarity = subtle connection = hard.

Empirical thresholds (paper vs. our measurements on 554 NYT puzzles):

Color	Paper Mean	Our Mean	Our Std
Yellow	0.40	0.389	0.089
Green	0.35	0.315	0.060
Blue	0.29	0.268	0.050
Purple	0.27	0.225	0.049

NYT color matches similarity rank only 43.9% of the time, confirming that difficulty has a knowledge/wordplay component embeddings cannot capture.

Stage 5: Deduplication

Each puzzle's 16-word set is compared against all 554 NYT puzzles. If more than 6 words overlap with any single NYT puzzle, it is flagged.

Pipeline Summary

Stage	Who	Why
Group creation	LLM	Invents categories + candidates
Word selection	MPNET	Picks most cohesive 4 from 8
Editor pass	LLM	Fixes inaccurate names
Color assignment	MPNET	Maps similarity to difficulty
Deduplication	Set math	Prevents copying NYT puzzles

4. Solver Algorithms

Three independent solvers verify that each puzzle has exactly one valid solution.

Solver 1: Embedding Solver (Greedy)

Source: 'Missed Connections' paper (arXiv:2404.11730).

Algorithm:

1. Encode all 16 words with MPNET
2. Compute 16x16 cosine similarity matrix
3. Score all $C(16,4) = 1,820$ possible groups by avg pairwise similarity
4. Sort by score descending
5. Greedily pick the highest-scoring non-overlapping groups (4 total)

Complexity: $O(1,820)$ group evaluations + sorting.

Runtime: ~0.019s per puzzle.

Accuracy: 2.0% full-puzzle solve rate.

Limitation: greedy selection can 'use up' key words in a wrong group, causing cascading errors. The solver never backtracks.

Solver 2: Clustering Solver (Beam Search)

Source: 'Deceptively Simple' paper (arXiv:2412.01621).

Group Similarity Score (G)

Group Similarity Score

$$G = 0.4 * I + 0.3 * s + 0.3 * V$$

$$I = -\text{SUM}(||e_i - \text{centroid}||^2) \quad (\text{negative k-means inertia})$$

$$s = \text{MIN over all } i < j \text{ of } \cos(e_i, e_j) \quad (\text{weakest link})$$

$$V = \text{mean}(P) / (1 + \text{var}(P)) \quad (\text{stability-weighted mean})$$

$$P = \{ \cos(e_i, e_j) : i < j \} \quad (\text{all 6 pairwise similarities})$$

Intuition for each component:

- I (inertia): How tightly clustered? Tighter = higher score.
- s (min similarity): A group is only as strong as its weakest pair.
- V (stability): Rewards consistent similarity. A group with all pairs at 0.3 scores higher than {0.5, 0.5, 0.5, 0.1, 0.1, 0.1} even if means are equal.

Penalty Score (P)

Penalty Score

$$P = (1/|R|) * \text{SUM over } r \text{ in } R \text{ of } \cos(\text{centroid}_C, r)$$

R = remaining words after removing group C

centroid_C = mean of group C 's embeddings

Lower penalty = better separation from remaining words

Beam Search

Algorithm:

1. Initialize beam with empty grouping (all 16 words remaining)
2. For each of 4 steps:
 - a. For each state in the beam, enumerate all $C(\text{remaining}, 4)$ groups
 - b. Score each: cumulative $\pm G - P$
 - c. Keep top 10 candidates (beam width)
3. Return the highest-scoring complete 4-group solution

Step sizes: Step 1: $10 \times 1,820 = 18,200$ evaluations.

Step 2: 10×495 . Step 3: 10×70 . Step 4: 10×1 .

Runtime: ~0.95s per puzzle (~50x slower than embedding solver).

Accuracy: 3.4% full-puzzle solve rate.

Solver 3: LLM Solver (Chain-of-Thought)

Source: 'Missed Connections' paper. Presents all 16 words to Claude with step-by-step reasoning instructions. Most human-like but expensive (API call per puzzle). Used as a tiebreaker only.

Solver Performance Comparison

Metric	Embedding	Clustering
Full solve rate	2.0% (11/554)	3.4% (19/554)
Avg groups correct	0.57 / 4	0.71 / 4
Yellow accuracy	24.5%	28.0%
Green accuracy	17.7%	22.6%
Blue accuracy	9.8%	13.0%
Purple accuracy	5.1%	7.6%
Runtime (554 puzzles)	10.6s	525s

Both solvers show the expected difficulty gradient. The clustering solver outperforms across all colors at ~50x the compute cost. They agreed on a full solve for only 7 puzzles (1.3%), confirming complementary approaches.

5. Roundtable Validation

Key Idea

This is the project's most original contribution. The papers each address generation OR solving; we combine them by using solver agreement as a quality gate for generated puzzles.

The Core Idea

A valid Connections puzzle must have exactly one correct solution. If two independent solvers, using completely different algorithms, both find the same grouping -- and that matches the intended answer -- then the puzzle is unambiguous.

If they disagree, the puzzle has competing interpretations and should be discarded.

Validation Logic

1. Run Embedding Solver on the 16 words -> solution S1
2. Run Clustering Solver on the 16 words -> solution S2
3. Normalize both to sets of frozensets (order-independent)
4. Check: $S1 == S2$? (same 4 groups?)
5. Check: do they match the intended answer?
6. Valid = agreement AND at least one solver got all 4 correct

Why Two Solvers?

The embedding solver (greedy, optimizes avg pairwise similarity) and clustering solver (beam search, optimizes $G - P$ composite) have different failure modes. When both converge to the same answer, it is strong evidence that the answer is unique.

6. Evaluation Metrics

Group Similarity Score (G): identical to the clustering solver's formula. Used as an independent quality metric.

Penalty Score (P): measures group-group separation.

Puzzle-level quality:

- Overall avg similarity: mean of s_avg across all 4 groups
- Similarity spread: $\max(s_avg) - \min(s_avg)$. Should be positive.
- Solver agreement rate: fraction of puzzles where solvers converge
- Category diversity: number of unique category names

7. System Architecture

Data Flow

LLM (8 candidates) -> MPNET (select best 4) -> Editor (LLM) -> Difficulty (MPNET)

|
v
Deduplication

|
v
Roundtable Validator
Agree? -> Accept / Discard

Dry-Run Mode

All LLM calls go through LLMClient, which checks dry_run. If true, MockLLMClient returns realistic hardcoded responses by detecting request type from prompt keywords. MPNET always runs for real.

Component	Dry-run	Real mode
Category ideas + pools	Mock	Claude API
Word selection (8->4)	Real MPNET	Real MPNET
Editor pass	Mock	Claude API
Difficulty colors	Real MPNET	Real MPNET
Embedding solver	Real MPNET	Real MPNET
Clustering solver	Real MPNET	Real MPNET
LLM solver	Mock	Claude API

8. Key Formulas Reference Card

All Formulas in One Place

COSINE SIMILARITY:

$$\cos(a, b) = (a \cdot b) / (||a|| * ||b||)$$

AVERAGE PAIRWISE SIMILARITY (4 words):

$$s_avg = (1/6) * \sum_{i < j} \cos(e_i, e_j)$$

GROUP SIMILARITY SCORE:

$$G = 0.4 * I + 0.3 * s + 0.3 * V$$

$$I = -(\sum ||e_i - \text{centroid}||^2) \quad [\text{neg. inertia}]$$

$$s = \min_{i < j} \cos(e_i, e_j) \quad [\text{weakest link}]$$

$$V = \text{mean}(P) / (1 + \text{var}(P)) \quad [\text{stability}]$$

PENALTY SCORE:

$$P = (1/|R|) * \sum_r \cos(\text{centroid}_C, r)$$

BEAM SEARCH OBJECTIVE:

$$\text{maximize } \sum_{\text{groups}} (G - P)$$

COLOR ASSIGNMENT:

sort by s_avg descending -> Yellow, Green, Blue, Purple

VALIDATION:

$$\text{valid} = (\text{solver1} == \text{solver2}) \text{ AND } (\text{any solver got } 4/4)$$

9. What Is Original vs. From the Literature

Component	Status	Source
Embedding solver	Reimplementation	Missed Connections (2404.11730)
Clustering solver	Reimplementation	Deceptively Simple (2412.01621)
False-group pipeline	Reimplementation	Making New Connections (2407.11240)
Color thresholds	Constants	Making New Connections Table 1
LLM solver (CoT)	Reimplementation	Missed Connections
Roundtable validator	NOVEL	Original to this project
Prompt templates	NOVEL	Original text
Mock/dry-run system	NOVEL	Original engineering
Web app	NOVEL	Original code
End-to-end integration	NOVEL	Papers do gen OR solving; we combine

All code is written from scratch. No code was copied from any reference repository.

10. References

-
- [1] Conneau et al., 'Making New Connections,' arXiv:2407.11240, 2024.
Generation pipeline, false groups, story injection, color thresholds.
- [2] Todd et al., 'Missed Connections,' arXiv:2404.11730, 2024.
Embedding solver, LLM solver baselines, CoT prompts.
- [3] Li et al., 'Deceptively Simple,' arXiv:2412.01621, 2024.
 $G = 0.4I + 0.3s + 0.3V$ formula, penalty score, beam search.
- [4] Patel et al., 'Connecting the Dots,' arXiv:2406.11012, 2024.
Category types, human evaluation methodology.